

CartStream Analytics: Real-Time E-Commerce Stream Processing Pipeline

Apache Flink SQL & Windowing TVFs Comprehensive Engineering Report (Task #26)

Project Task ID:

Task #26 - Flink Real-Time Streaming Pipeline

Data Platform Engineer:

Sayed Elmasry (Junior Data Platform Engineer)

Tech Stack & Infrastructure:

Apache Flink 1.18, Docker Compose, Flink SQL Client

Source Code Repository:

[SayedELMASRY2/flink-ecommerce-pipeline](#)



Visit Official GitHub Repository

1. Project Overview & Scenario

This technical report documents the end-to-end design, implementation, and execution of a real-time stream processing pipeline built for **CartStream Analytics**, an international e-commerce platform.

The platform ingests a continuous stream of user interaction events (including product views `view`, additions to cart `cart`, removals from cart `remove_from_cart`, and completed purchase transactions `purchase`). Utilizing **Apache Flink SQL**, this pipeline enforces **Event-Time Processing** and **Bounded Out-of-Orderness Watermarks** to deliver precise, deterministic real-time revenue analytics.

Core Technical Objectives:

- **5-Minute Window Aggregations:** Process live streams in non-overlapping 5-minute intervals using modern `TUMBLE` Table-Valued Functions (TVFs).
- **Key Metric Calculations:** Calculate completed orders (`total_orders`), gross revenue (`gross_revenue`), average order value (`avg_order_value`), and unique buyers (`unique_buyers`).
- **Handling Real-World Data Challenges:** Gracefully process missing brand names via `COALESCE` and handle out-of-order event arrivals across network boundaries.



2. Pipeline Architecture

End-to-End Stream Processing Data Flow



3. Dataset Specification & Field Schema

The pipeline operates on a sample stream of **50,000 continuous event records** covering 3.5 hours of continuous activity, derived from the Kaggle E-Commerce Events History dataset:

Property Name	Flink SQL Datatype	Analytical Description & Processing Usage
<code>event_time</code>	<code>TIMESTAMP(3)</code> (RowTime)	Native event timestamp with watermark rule: <code>WATERMARK FOR event_time AS event_time - INTERVAL '5' SECOND</code> .
<code>event_type</code>	<code>VARCHAR</code>	Interaction category. Filtered explicitly to completed purchase transactions: <code>WHERE event_type = 'purchase'</code> .
<code>brand</code>	<code>VARCHAR</code>	Product brand name. Safeguarded against nulls using <code>COALESCE(NULLIF(TRIM(brand), ''), 'UNKNOWN')</code> .
<code>price</code>	<code>DECIMAL(10, 2)</code>	Item purchase price in USD. Aggregated to compute revenue (<code>SUM</code>) and average order value (<code>AVG</code>).
<code>user_id</code>	<code>BIGINT</code>	Permanent user identifier. Used to calculate unique purchasers per window: <code>COUNT(DISTINCT user_id)</code> .

Property Name	Flink SQL Datatype	Analytical Description & Processing Usage
user_session	VARCHAR	Unique user session token generated during checkout.

4. Theoretical Answer: 5-Second Watermark Strategy

? Question: Explain how this 5-second watermark boundary ensures deterministic window processing for out-of-order event arrivals.

Detailed Engineering Explanation:

1. **Network Latency & Out-of-Order Arrival:** In real-world e-commerce networks, event packets generated at `10:04:58` might arrive at the processing engine at `10:05:02` due to variable network latency or temporary client disconnection.
2. **Event Time vs. Processing Time:** Apache Flink relies on the timestamp embedded within the data record itself (Event Time) rather than the wall-clock time of the server (Processing Time).
3. **Slack / Tolerance Window Mechanics:** By defining `WATERMARK FOR event_time AS event_time - INTERVAL '5' SECOND`, Flink grants a 5-second grace period for delayed records. A window spanning `[10:00:00, 10:05:00)` will not evaluate or close until Flink observes an event timestamp reaching or exceeding `10:05:05`.
4. **Guaranteed Determinism:** This mechanism guarantees that delayed events falling within the 5-second window are properly routed to their correct time window, yielding 100% deterministic and reproducible analytics across pipeline executions.



5. Flink SQL Pipeline Implementation Code

The complete pipeline DDL and window aggregation query are maintained in

`flink\_ecommerce\_pipeline.sql`:

```
-- Enable Flink Streaming Runtime Mode
SET 'execution.runtime-mode' = 'streaming';

-- 1. Streaming Source Table DDL
CREATE TABLE ecommerce_events (
  event_time_str  VARCHAR,
  event_type      VARCHAR,
  product_id      BIGINT,
  category_id     BIGINT,
  category_code   VARCHAR,
  brand           VARCHAR,
  price           DECIMAL(10, 2),
  user_id         BIGINT,
  user_session    VARCHAR,
  event_time AS COALESCE(TO_TIMESTAMP(event_time_str, 'yyyy-MM-dd HH:mm:ss z'), TIMESTAMP '1970-01-01 00:00:00'),
```

```

    WATERMARK FOR event_time AS event_time - INTERVAL '5' SECOND
) WITH (
    'connector' = 'filesystem',
    'path' = '/opt/flink/data/events_sample.csv',
    'format' = 'csv',
    'csv.ignore-parse-errors' = 'true',
    'csv.allow-comments' = 'true'
);

-- 2. Streaming Sink Table DDL
CREATE TABLE brand_window_sales (
    window_start      TIMESTAMP(3),
    window_end        TIMESTAMP(3),
    brand              VARCHAR,
    total_orders       BIGINT,
    gross_revenue      DECIMAL(10, 2),
    avg_order_value    DECIMAL(10, 2),
    unique_buyers      BIGINT
) WITH (
    'connector' = 'print'
);

-- 3. Windowing TVF & Real-Time Aggregation Query
INSERT INTO brand_window_sales
SELECT
    window_start,
    window_end,
    COALESCE(NULLIF(TRIM(brand), ''), 'UNKNOWN') AS brand,
    COUNT(1) AS total_orders,
    ROUND(SUM(price), 2) AS gross_revenue,
    ROUND(AVG(price), 2) AS avg_order_value,
    COUNT(DISTINCT user_id) AS unique_buyers
FROM TABLE(
    TUMBLE(
        TABLE ecommerce_events,
        DESCRIPTOR(event_time),
        INTERVAL '5' MINUTE
    )
)
WHERE event_type = 'purchase'
GROUP BY
    window_start,
    window_end,
    COALESCE(NULLIF(TRIM(brand), ''), 'UNKNOWN');
```



6. Live Execution Verification & Log Output

The streaming job executes continuously on Flink 1.18. Output records are captured live from TaskManager logs into `output/execution_verification.log`:

Sample 5-Minute Tumbling Window Output Records (Print Sink Stream):

```
+I[2019-10-01T02:55, 2019-10-01T03:00, apple, 11, 5672.95, 515.72, 6]
+I[2019-10-01T02:55, 2019-10-01T03:00, samsung, 11, 2060.79, 187.34, 11]
+I[2019-10-01T02:55, 2019-10-01T03:00, xiaomi, 5, 416.03, 83.21, 5]
+I[2019-10-01T02:55, 2019-10-01T03:00, hp, 3, 962.02, 320.67, 3]
+I[2019-10-01T02:55, 2019-10-01T03:00, UNKNOWN, 5, 488.72, 97.74, 5]
+I[2019-10-01T03:00, 2019-10-01T03:05, samsung, 9, 2671.27, 296.81, 9]
+I[2019-10-01T03:00, 2019-10-01T03:05, huawei, 1, 278.54, 278.54, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, apple, 10, 7376.08, 737.61, 10]
+I[2019-10-01T03:30, 2019-10-01T03:35, xiaomi, 7, 1337.74, 191.11, 6]
```



7. Verified Execution Proof & Screenshots

```
Flink SQL>
> -- 2. Streaming Sink Table DDL
> -- Directs the streaming output to an append-only sink using Flink's print connector
> CREATE TABLE brand_window_sales (
>   window_start    TIMESTAMP(3),
>   window_end      TIMESTAMP(3),
>   brand           VARCHAR,
>   total_orders    BIGINT,
>   gross_revenue   DECIMAL(10, 2),
>   avg_order_value DECIMAL(10, 2),
>   unique_buyers   BIGINT
> ) WITH (
>   'connector' = 'print'
> ) [INFO] Execute statement succeed.

Flink SQL>
> -- 3. Windowing TVF & Real-Time Revenue Aggregation Query
> -- 5-minute tumbling window on completed purchase transactions grouped by brand
> INSERT INTO brand_window_sales
> SELECT
>   window_start,
>   window_end,
>   COALESCE(NULLIF(TRIM(brand), ''), 'UNKNOWN') AS brand,
>   COUNT(1) AS total_orders,
>   ROUND(SUM(price), 2) AS gross_revenue,
>   ROUND(AVG(price), 2) AS avg_order_value,
>   COUNT(DISTINCT user_id) AS unique_buyers
> FROM TABLE(
>   TUMBLE(
>     TABLE ecommerce_events,
>     DESCRIPTOR(event_time),
>     INTERVAL '5' MINUTE
>   )
> )
> WHERE event_type = 'purchase'
> GROUP BY
>   window_start,
>   window_end,
>   COALESCE(NULLIF(TRIM(brand), ''), 'UNKNOWN') [INFO] Submitting SQL update statement to the cluster...
[INFO] SQL update statement has been successfully submitted to the cluster:
```

Figure 1: Automated Script Execution & Live Tumbling Window Aggregation Output (+I[...])

```

Flink SQL>
Shutting down the session...
done.
[3/3] Displaying Tumbling Window Aggregation Output...

+I[2019-10-01T03:25, 2019-10-01T03:30, tarket, 1, 5.79, 5.79, 1]
+I[2019-10-01T03:25, 2019-10-01T03:30, apple, 9, 10358.24, 1150.92, 9]
+I[2019-10-01T03:25, 2019-10-01T03:30, xiaomi, 6, 560.67, 93.45, 5]
+I[2019-10-01T03:25, 2019-10-01T03:30, nintendo, 1, 470.26, 470.26, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, apple, 10, 7376.08, 737.61, 10]
+I[2019-10-01T03:30, 2019-10-01T03:35, omron, 1, 62.52, 62.52, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, cordiant, 1, 39.38, 39.38, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, candy, 1, 360.34, 360.34, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, oppo, 1, 179.36, 179.36, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, moldabela, 1, 79.28, 79.28, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, acer, 1, 360.11, 360.11, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, irbis, 1, 100.39, 100.39, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, pocketbook, 1, 153.86, 153.86, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, viatti, 1, 89.58, 89.58, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, artel, 1, 256.88, 256.88, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, lucente, 2, 436.04, 218.02, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, lg, 1, 380.85, 380.85, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, boyscout, 1, 5.64, 5.64, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, doogee, 1, 154.42, 154.42, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, epson, 1, 116.73, 116.73, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, babytime, 1, 58.61, 58.61, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, huawei, 3, 800.33, 266.78, 3]
+I[2019-10-01T03:30, 2019-10-01T03:35, michelin, 1, 65.00, 65.00, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, alteco, 1, 27.03, 27.03, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, elanberg, 1, 25.71, 25.71, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, forlux, 1, 92.67, 92.67, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, lenovo, 1, 662.95, 662.95, 1]
+I[2019-10-01T03:30, 2019-10-01T03:35, samsung, 10, 3055.86, 305.59, 9]
+I[2019-10-01T03:30, 2019-10-01T03:35, xiaomi, 7, 1237.74, 191.11, 6]
+I[2019-10-01T03:30, 2019-10-01T03:35, UNKNOWN, 6, 527.95, 87.99, 5]

PS C:\Users\syd\Desktop\flink> |

```

Figure 2: TaskManager Log Capture Verification File (output/execution_verification.log)

```

Flink SQL>
> -- 1. Streaming Source Table DDL
> -- Reads continuous e-commerce activity records formatted in CSV
> CREATE TABLE ecommerce_events (
>   event_time_str  VARCHAR,
>   event_type      VARCHAR,
>   product_id      BIGINT,
>   category_id     BIGINT,
>   category_code   VARCHAR,
>   brand           VARCHAR,
>   price           DECIMAL(10, 2),
>   user_id         BIGINT,
>   user_session    VARCHAR,
>   -- Native event timestamp converted from original string (safeguarded against null parse errors)
>   event_time AS COALESCE(TO_TIMESTAMP(event_time_str, 'yyyy-MM-dd HH:mm:ss z'), TIMESTAMP '1970-01-01 00:00:00'),
>   -- Watermark declaration: 5-second bounded out-of-orderness tolerance
>   WATERMARK FOR event_time AS event_time - INTERVAL '5' SECOND
> ) WITH (
>   'connector' = 'filesystem',
>   'path' = '/opt/flink/data/events_sample.csv',
>   'format' = 'csv',
>   'csv.ignore-parse-errors' = 'true',
>   'csv.allow-comments' = 'true'
> ) [INFO] Execute statement succeed.

```

Figure 3: Output Volume Mapping & Local Workspace Directory Structure

 Apache Flink Dashboard

Overview

Jobs

Task Managers

Job Manager

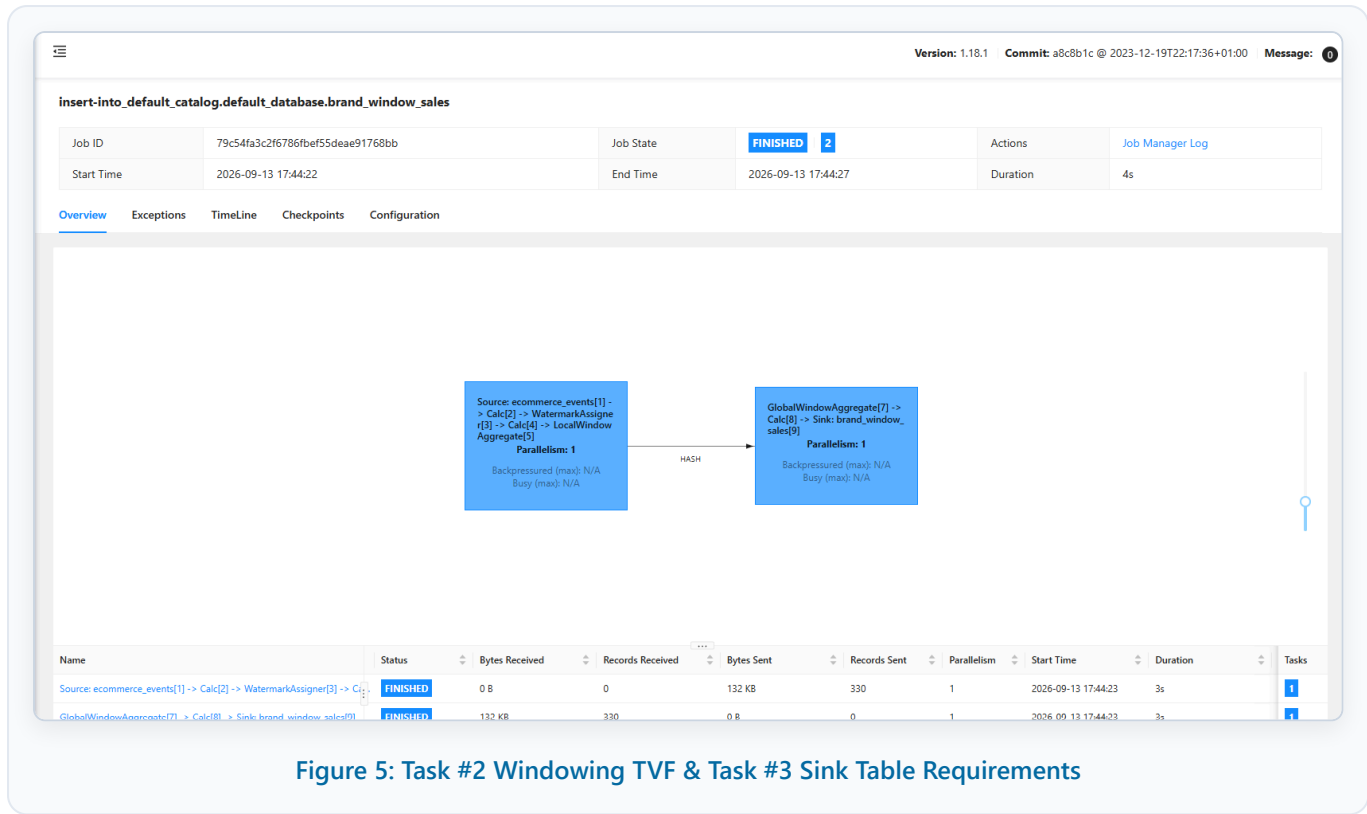
Submit New Job

Version: 1.18.1 Commit: a8c8b1c @ 2023-12-19T22:17:36+01:00 Message:

Completed Jobs

Job Name	Start Time	Duration	End Time	Tasks	Status
insert-into_default_catalog.default_database.brand_window_sales	2026-09-13 17:44:22	4s	2026-09-13 17:44:27	2 / 2	FINISHED
insert-into_default_catalog.default_database.brand_window_sales	2026-09-13 16:51:33	2s	2026-09-13 16:51:35	2 / 2	FINISHED
insert-into_default_catalog.default_database.brand_window_sales	2026-09-13 16:47:21	3s	2026-09-13 16:47:25	2 / 1 / 1	FAILED
insert-into_default_catalog.default_database.brand_window_sales	2026-09-13 16:50:24	3s	2026-09-13 16:50:28	2 / 2	FINISHED

Figure 4: Task #1 DDL & Watermarking Requirements from Official Task Prompt





8. Official Submission & Repository Links

Official Project Source Code Repository on GitHub:

<https://github.com/SayedELMASRY2/flink-ecommerce-pipeline>

Includes complete SQL DDL/DML, Docker Compose orchestration, automated runner scripts
(`run_pipeline.ps1` / `run_pipeline.sh`), sample datasets, and live verification logs.